

PROJET MACHINE LEARNING

Prédiction du prix des voitures d'occasion

Un pipeline complet, de la compréhension métier au déploiement sur Streamlit

Tâche : régression supervisée · Modèle final : XGBoost · $R^2 \approx 0,90$

1. Compréhension de la problématique

Le marché de la voiture d'occasion est vaste mais peu transparent. Deux véhicules quasiment identiques peuvent être affichés à des prix sensiblement différents selon le vendeur, la région ou simplement son expérience. Fixer le bon prix est donc un vrai problème : trop élevé, l'annonce ne trouve pas preneur ; trop bas, le vendeur y perd. L'objectif de ce projet est d'aider à estimer automatiquement un prix de marché « juste » à partir des caractéristiques d'un véhicule.

Les données proviennent d'annonces de voitures d'occasion publiées sur eBay (Allemagne). Chaque ligne décrit un véhicule : marque, modèle, type, année d'immatriculation, kilométrage, puissance, type de boîte, carburant, état (dommage non réparé) et, bien sûr, le prix demandé.

Du problème métier à la tâche ML

La variable à prédire, **price**, est une grandeur continue exprimée en euros. Il s'agit donc sans ambiguïté d'un problème de **régression supervisée** (et non de classification). On dispose en entrée des caractéristiques du véhicule (X) et en sortie du prix observé (y). La qualité des modèles est jugée principalement avec trois indicateurs : le coefficient de détermination R^2 (part de la variance expliquée), l'erreur absolue moyenne MAE (en euros, directement interprétable côté métier) et l'erreur quadratique moyenne MSE / RMSE (qui pénalise davantage les grosses erreurs).

État de l'art

La prédiction sur données *tabulaires* est un sujet très étudié. On distingue plusieurs familles d'approches :

- **Modèles linéaires** (régression linéaire, Ridge, Lasso) : simples et interprétables, mais ils captent mal les relations non linéaires — l'effet de l'âge ou du kilométrage sur le prix n'a rien de linéaire.
- **k plus proches voisins (KNN)** : intuitif et non paramétrique, mais sensible à l'échelle des variables et coûteux quand le jeu de données grandit.
- **Arbres de décision** : capturent les non-linéarités et les interactions, mais sur-apprennent facilement lorsqu'ils sont utilisés seuls.
- **Méthodes d'ensemble** : l'état de l'art actuel sur ce type de données. Le *bagging* (Random Forest — Breiman, 2001 ; Extra Trees) réduit la variance ; le *boosting* (Gradient Boosting — Friedman, 2001 ; puis XGBoost — Chen & Guestrin, 2016 ; LightGBM ; CatBoost — Prokhorenkova et al., 2018) domine régulièrement les compétitions Kaggle.
- **Réseaux de neurones profonds** : très performants sur l'image et le texte, mais sur des données tabulaires de taille moyenne ils sont souvent égalés, voire dépassés, par le gradient boosting (Grinsztajn et al., 2022).
- **Explicabilité (XAI)** : SHAP (Lundberg & Lee, 2017) s'est imposé pour mesurer la contribution de chaque variable à la prédiction.

Ce constat a guidé notre stratégie : tester un large éventail de modèles, mais miser en priorité sur les méthodes d'ensemble à base d'arbres.

2. Analyse exploratoire des données (EDA)

Le jeu de données brut compte environ vingt colonnes de types variés : numériques (price, powerPS, kilometer, yearOfRegistration...), catégorielles (brand, model, vehicleType, fuelType, gearbox...) et temporelles (dateCrawled, dateCreated, lastSeen). La variable cible est **price**.

Principaux constats

- **Valeurs manquantes** importantes sur plusieurs catégorielles : notRepairedDamage ($\approx 19\%$), vehicleType ($\approx 10\%$), fuelType ($\approx 9\%$), model et gearbox ($\approx 5\%$).
- **Valeurs aberrantes** très marquées sur le prix : min à 0 et max à plus de 2 milliards, avec une moyenne ($\approx 16\,600$) très éloignée de la médiane ($\approx 2\,950$). La distribution est donc fortement asymétrique (skewed) et polluée par des outliers.
- **Zéros suspects** sur powerPS ($\approx 11\%$) et monthOfRegistration ($\approx 10\%$).
- L'**année d'immatriculation** contient des valeurs impossibles (années très anciennes ou futures).
- Le **kilométrage** est discrétisé et plafonné à 150 000 km : peu de variabilité, mais exploitable tel quel.
- Des **valeurs catégorielles en allemand** (manuell, automatik, ja, nein, benzin...) à traduire pour la lisibilité.

Corrélations et tendances

L'étude des corrélations (Pearson, Spearman, Kendall) montre que le prix est surtout lié à l'année d'immatriculation (donc à l'âge) et à la puissance (powerPS). À l'inverse, le mois d'immatriculation n'a aucune corrélation notable avec le prix : cette variable sera abandonnée. Les graphiques (histogrammes, KDE, diagrammes en barres) confirment ces tendances : un véhicule plus récent et plus puissant se vend logiquement plus cher.

Conclusion de l'EDA : les données sont riches mais nécessitent un nettoyage sérieux (manquants, outliers, traduction, bornage des plages) avant toute modélisation.

3. Prétraitement des données

Le prétraitement a été pensé pour être à la fois fidèle à l'analyse exploratoire et **reproductible en production**. Les étapes retenues :

- **Colonnes supprimées** : dateCrawled, lastSeen, dateCreated, name, seller, offerType, nrOfPictures, postalCode et monthOfRegistration — soit parce qu'elles sont constantes (presque toutes les annonces sont privées), soit sans lien avec le prix.
- **Bornage des plages** : année d'immatriculation entre 1980 et 2023, prix dans [200 €, 20 000 €] (cette borne élimine l'essentiel des outliers tout en conservant le gros du marché), puissance dans [0, 1000] ch.
- **Traduction** des modalités allemandes en anglais (manuell → Manual, ja → Yes, benzin → Petrol, etc.).

- **Variable dérivée** : création de l'âge du véhicule ($\text{age} = 2023 - \text{année}$), puis suppression de `yearOfRegistration`.
- **Valeurs manquantes** : suppression des lignes incomplètes — le volume restant est suffisant pour entraîner les modèles.
- **Encodage** des catégorielles (`OrdinalEncoder`) et **normalisation Z-score** (`StandardScaler`) des numériques `powerPS`, `kilometer` et `age`.

Sélection des variables (feature selection)

Au terme du nettoyage, on conserve **dix variables explicatives** : `vehicleType`, `gearbox`, `model`, `fuelType`, `brand`, `notRepairedDamage`, `abtest`, `powerPS`, `kilometer` et `age`. Le choix s'appuie sur l'analyse des corrélations (suppression du `mois`, sans lien avec le prix) et sur le bon sens métier. Cette sélection sera confortée a posteriori par l'analyse SHAP, qui désigne `age` et `powerPS` comme les variables les plus déterminantes.

Un point important sur la fuite de données

Lors de l'exploration, l'encodage et la normalisation ont d'abord été appliqués sur l'ensemble du jeu. Pour la version déployable, nous avons corrigé ce point en plaçant l'encodage et la mise à l'échelle **à l'intérieur d'un pipeline scikit-learn**, ajusté uniquement sur les données d'entraînement. On évite ainsi toute fuite et on garantit que les mêmes transformations seront appliquées en production.

4. Construction des modèles

Conformément à l'objectif de tester au moins dix algorithmes, nous avons couvert un large spectre, du réseau de neurones aux méthodes d'ensemble, le tout évalué par validation croisée (5 plis) sur R^2 , MAE et MSE.

Réseau de neurones profond (DNN)

Un DNN (couches denses 32-64-128-64-32, activation ReLU, Dropout 0,2, sortie linéaire, perte MSE, optimiseur Adam, EarlyStopping et ordonnanceur de learning rate) a été entraîné sur 50 époques. Résultat : un R^2 d'environ 0,80 — honorable, mais en deçà de nos attentes pour ce type de données tabulaires.

Modèles classiques et d'ensemble

Modèle	Type	R^2 (test, approx.)
XGBoost	Boosting	$\approx 0,90$
CatBoost	Boosting	$\approx 0,89$
Extra Trees	Bagging	$\approx 0,88$
Random Forest	Bagging	$\approx 0,88$
KNN	Voisinage	$\approx 0,83$
Decision Tree	Arbre seul	$\approx 0,82$
DNN	Réseau de neurones	$\approx 0,80$

AdaBoost	Boosting	négatif (échec d'ajustement)
Ensembles supplémentaires : moyenne simple, moyenne pondérée, vote (VotingRegressor) et stacking (StackingRegressor avec Gradient Boosting comme méta-modèle). En comptant le DNN, les sept régresseurs et les quatre techniques d'ensemble, ce sont bien plus de dix approches qui ont été testées.		

Lecture des résultats : XGBoost offre la meilleure performance, suivi de près par CatBoost et les forêts. Le cas d'AdaBoost (R^2 négatif) n'est pas une erreur : mathématiquement, un R^2 peut être négatif lorsque le modèle prédit moins bien que la simple moyenne — ici AdaBoost n'a tout simplement pas réussi à s'ajuster. Les techniques d'ensemble (vote, stacking) n'ont pas dépassé XGBoost sur le R^2 , même si elles ont légèrement réduit la MAE.

Explicabilité (XAI) avec SHAP

Pour ne pas garder une « boîte noire », nous avons appliqué SHAP à XGBoost et CatBoost. Les deux racontent la même histoire : **age** et **powerPS** sont les variables qui pèsent le plus dans la prédiction du prix, ce qui est cohérent avec l'intuition métier et valide a posteriori notre sélection de variables.

5. Tuning des candidats et choix du modèle final

Sur la base des résultats précédents, nous avons retenu **trois modèles candidats** : XGBoost (meilleur R^2), CatBoost (très proche et peu sensible au réglage) et Random Forest (baseline d'ensemble robuste). Les autres ont été écartés : AdaBoost (R^2 négatif), arbre seul (sur-apprentissage), KNN (passage à l'échelle).

Chaque candidat a été optimisé avec **RandomizedSearchCV** (recherche aléatoire, 3 plis, métrique R^2) sur ses hyperparamètres clés — nombre d'arbres, profondeur, learning rate, sous-échantillonnage. Les performances avant / après réglage ont été comparées :

Modèle candidat	R^2 avant	R^2 après	Gain
XGBoost	≈ 0,895	≈ 0,905	+ léger
CatBoost	≈ 0,888	≈ 0,898	+ léger
Random Forest	≈ 0,878	≈ 0,881	≈ stable

Valeurs indicatives issues de la validation croisée du notebook ; les écarts exacts dépendent du tirage aléatoire.

Le tuning apporte un gain net surtout pour les deux modèles de boosting (réglage du nombre d'arbres, de la profondeur et du learning rate). Random Forest, lui, était déjà proche de son optimum par défaut.

Modèle final et vérification du sur-apprentissage

Le modèle final retenu est **XGBoost**. Pour s'assurer qu'il généralise bien, on compare le R^2 sur l'entraînement et sur le test : l'écart reste faible, ce qui écarte un sur-apprentissage marqué. Le modèle atteint un R^2 de l'ordre de 0,90 sur le jeu de test, avec une MAE de quelques centaines d'euros — une erreur raisonnable au regard des prix considérés (200 à 20 000 €). C'est cette MAE qui servira de niveau de confiance dans l'application.

6. Déploiement

Le pipeline complet — **prétraitement (encodage + normalisation) et modèle XGBoost réunis dans un seul objet scikit-learn** — est sérialisé avec joblib (`model_pipeline.joblib`). On y joint quelques métadonnées utiles à l'application : la MAE de test (pour l'intervalle de confiance), les modalités possibles de chaque variable catégorielle (menus déroulants) et les plages des variables numériques (curseurs).

Une **application Streamlit** (`app.py`) charge ce pipeline et propose une interface simple : l'utilisateur saisit les caractéristiques de son véhicule, l'application affiche les données saisies, puis renvoie le prix estimé accompagné d'un **intervalle de confiance** ($\text{prix} \pm \text{MAE}$). S'agissant d'une régression, ce niveau de confiance prend la forme d'une fourchette de prix plutôt que d'une probabilité.

Le dépôt GitHub regroupe l'ensemble des livrables : `requirements.txt`, `README.md` (avec description, lien vers l'application déployée et captures d'écran), le notebook complet, le présent rapport, le script `app.py` et le pipeline sérialisé. L'application est publiée sur Streamlit Cloud.

7. Conclusion

Ce projet a permis de dérouler un pipeline ML complet, de la compréhension du problème métier jusqu'au déploiement. Après une analyse exploratoire qui a mis en évidence des valeurs manquantes et des outliers importants, un prétraitement rigoureux a préparé des données propres et exploitables. Plus de dix algorithmes ont été comparés ; le gradient boosting s'est révélé le plus adapté, devançant aussi bien le réseau de neurones que les ensembles par vote ou stacking.

Le modèle final, un XGBoost réglé par recherche d'hyperparamètres, atteint un R^2 proche de 0,90 sans sur-apprentissage notable, et l'analyse SHAP a confirmé le rôle central de l'âge et de la puissance. Enfin, l'ensemble du prétraitement et du modèle a été encapsulé dans un pipeline unique, sérialisé puis exposé via une application Streamlit interactive.

Pistes d'amélioration : enrichir les données (récupérer les modalités manquantes par scraping), tester une transformation logarithmique du prix pour mieux gérer son asymétrie, ou encore comparer avec LightGBM et un réglage plus poussé via une recherche bayésienne.